



|                         |                                                                    |
|-------------------------|--------------------------------------------------------------------|
| <b>NAME:</b>            | <b>MUHAMMAD HASNAT FAKHAR</b>                                      |
| <b>REGISTRATION ID:</b> | <b>232436</b>                                                      |
| <b>CLASS / SECTION:</b> | <b>BSCS (evening)-F23-C</b>                                        |
| <b>DATE:</b>            | <b>16-September-2026</b>                                           |
| <b>Course-Code</b>      | <b>Introduction to Parallel &amp; Distributed Computing-CS426L</b> |
| <b>Lab 02</b>           | <b>Hardware Topology, Threads, and Flynn's Taxonomy</b>            |
| <b>Submitted to:</b>    | <b>Mr. Nauman Zafar Hashmi</b>                                     |

## Lab Task 02: One Thread Per Detected Core

### 5.1 Detecting the Number of Cores in C#

.NET provides the `Environment.ProcessorCount` property to determine the number of logical processors available to the current process.

```
// detect automatically the logical processors
int numCores = Environment.ProcessorCount;
```

On the laboratory machine used for this experiment, the program detected **12 logical processors**.

### 5.2 Threading Essentials in C#

```
1  using System;
2  using System.Threading;
3
4  class Program
5  {
6      // worker function indicating the work performed by each thread
7      static void Worker(int threadNumber)
8      {
9          Console.WriteLine("Thread " + threadNumber + ": running");
10     }
11
12     static void Main()
13     {
14         // detect automatically the logical processors
15         int numCores = Environment.ProcessorCount;
16
17         Console.WriteLine("Detected logical cores: " + numCores);
18
19         // dynamically allocating a thread array of core size
20         Thread[] threads = new Thread[numCores];
21
22         // creating and starting threads
23         for (int i = 0; i < numCores; i++)
24         {
25             // giving each thread its own number
26             int threadNumber = i;
27
28             // creating a new thread that will execute through Worker()
29             threads[i] = new Thread(() => Worker(threadNumber));
30
31             threads[i].Start();
32         }
33
34         // joining all threads
35         for (int i = 0; i < numCores; i++)
36         {
37             // the final message will appear only after every thread has completed
38             threads[i].Join();
39         }
40     }
41 }
```

## 5.4 Suggested Program Skeleton (Program.cs)

CoreThreads >  Program.cs

```
1  using System;
2  using System.Threading;
3
4  class Program
5  {
6      // worker function indicating the work performed by each thread
7      static void Worker(int threadNumber)
8      {
9          Console.WriteLine("Thread " + threadNumber + ": running");
10     }
11
12     static void Main()
13     {
14         // detect automatically the logical processors
15         int numCores = Environment.ProcessorCount;
16
17         Console.WriteLine("Detected logical cores: " + numCores);
18
19         // dynamically allocating a thread array of core size
20         Thread[] threads = new Thread[numCores];
21
22         // creating and starting threads
23         for (int i = 0; i < numCores; i++)
24         {
25             // giving each thread its own number
26             int threadNumber = i;
27
28             // creating a new thread that will execute through Worker()
29             threads[i] = new Thread(() => Worker(threadNumber));
30
31             threads[i].Start();
32         }
33
34         // joining all threads
35         for (int i = 0; i < numCores; i++)
36         {
37             // the final message will appear only after every thread has completed
38             threads[i].Join();
39         }
40
41         // displaying the total number of threads created
42         Console.WriteLine("All " + numCores + " threads completed.");
43         Console.WriteLine("Total threads created: " + numCores);
44     }
45 }
```

### 5.5 Optional: Reporting the Actual CPU a Thread Runs On

```
47 static void Worker(object? arg)
48 {
49     long id = (long)arg!;
50     int cpu = Thread.CurrentProcessorId();
51
52     Console.WriteLine($"Thread {id}: running on logical CPU {cpu}");
53 }
```

Build succeeded in 2.05

PS C:\Users\232460\Desktop\CoreThreads\CoreThreads> dotnet run

Detected logical cores: 12

Thread 5: running on logical CPU 4

Thread 2: running on logical CPU 0

Thread 6: running on logical CPU 9

Thread 7: running on logical CPU 11

Thread 1: running on logical CPU 0

Thread 0: running on logical CPU 2

Thread 8: running on logical CPU 10

Thread 3: running on logical CPU 0

Thread 4: running on logical CPU 4

Thread 9: running on logical CPU 4

Thread 10: running on logical CPU 11

Thread 11: running on logical CPU 7

All 12 threads completed.

Total threads created: 12

PS C:\Users\232460\Desktop\CoreThreads\CoreThreads> █

## 5.6 Compilation and Execution

```
PS C:\Users\232460\Desktop\CoreThreads\CoreThreads> dotnet run
Detected logical cores: 12
Thread 0: running
Thread 1: running
Thread 2: running
Thread 3: running
Thread 4: running
Thread 5: running
Thread 6: running
Thread 7: running
Thread 8: running
Thread 9: running
Thread 10: running
Thread 11: running
All 12 threads completed.
Total threads created: 12
PS C:\Users\232460\Desktop\CoreThreads\CoreThreads> dotnet run
Detected logical cores: 12
Thread 0: running
Thread 1: running
Thread 2: running
Thread 3: running
Thread 4: running
Thread 5: running
Thread 6: running
Thread 7: running
Thread 8: running
Thread 9: running
Thread 10: running
Thread 11: running
All 12 threads completed.
Total threads created: 12
```

## 5.7 Result

| Run | Detected Core | Threads Created | All Thread Completed |
|-----|---------------|-----------------|----------------------|
| 1   | 12            | 12              | Y                    |
| 2   | 12            | 12              | Y                    |

## 5.8 Observation Questions

### Question 1

**Did the program create exactly the number of threads reported by `Environment.ProcessorCount`? How does this compare with the total logical processor count recorded in Task 1?**

### Answer

Yes. The program detected 12 logical processors using `Environment.ProcessorCount` and created exactly 12 worker threads.

The results were:

Detected logical cores: 12

Total threads created: 12

Therefore:

Logical processors detected = 12

Threads created = 12

If the `CPU(s)` value in the Task 1 `lscpu` output is also 12, then the .NET result matches the Linux topology information.

---

### Question 2

**If you implemented the optional `Thread.GetCurrentProcessorId()` extension, did every thread run on a distinct logical core, or did the operating system schedule more than one thread onto the same core at some point?**

### Answer

The optional `Thread.GetCurrentProcessorId()` feature was not used in the provided experiment, so the exact logical CPU assignment of each thread cannot be determined from the recorded output.

Even when this method is used, creating 12 threads does not guarantee that each thread will permanently remain on a different logical processor. The operating system scheduler controls thread placement and may move threads between logical processors.

Therefore, a thread may execute on different logical processors at different times.

---

### Question 3

**What would happen if you created significantly more threads than the number of available logical cores? Would the program still function correctly?**

#### Answer

The program would normally continue to function correctly if more threads were created than the number of available logical processors.

For example, if the machine has 12 logical processors but the program creates 24 threads, the operating system must schedule those 24 threads using the available 12 logical processors.

This can result in:

- Multiple threads sharing the same logical processor over time.
- Increased context switching.
- Increased scheduling overhead.
- Greater CPU resource contention.
- Potentially longer execution time.

Therefore, having more threads than logical processors is possible, but more threads do not automatically result in better performance.

---

### Lab Task 03:Classifying the Machine Using Flynn's Taxonomy

#### 6.1 Flynn's Taxonomy

Flynn's Taxonomy is a classification system used to describe computer architectures according to the number of instruction streams and data streams being processed.

For a modern multicore CPU, the complete processor is generally classified as **MIMD**, while individual cores may also provide **SIMD capabilities** through vector instruction-set extensions.

---

#### 6.2 Required Steps

##### **Step 1: Identify the Number of Cores and Logical Processors**

According to the hardware information available for the laboratory machine:

- **Physical Cores:** 12
- **Logical Processors:** 12
- **Threads created in Task 2:** 12

## Step 2: Analyze SIMD Support

The `lscpu` command provides CPU feature information under the **Flags** field.

The relevant flags should be checked for features such as:

- `sse`
- `sse2`
- `sse4_1`
- `sse4_2`
- `avx`
- `avx2`
- `avx512`
- `neon`

The exact SIMD extensions supported by the laboratory machine should be copied from the `lscpu` output.

### SIMD flags from Task 1:

These flags indicate which SIMD/vector instruction-set extensions are supported by the processor.

---

## Step 3: Primary Classification

At the level of the complete multicore machine, the primary Flynn classification is:

### MIMD — Multiple Instruction, Multiple Data

The reason is that the machine contains multiple processing units capable of executing independent instruction streams. The Task 2 experiment demonstrated this capability by creating multiple independent C# threads.

The 12 logical processors allow multiple threads to execute concurrently. Therefore, the complete multicore system is classified as **MIMD**.

---

### Step 4: SIMD Capability

Although the complete machine is classified as **MIMD**, individual CPU cores may contain SIMD execution units.

Therefore, there are two different levels of classification:

- **Whole multicore machine:** MIMD
- **SIMD capability within individual cores:** SIMD operations may be supported through CPU instruction-set extensions such as SSE or AVX.

The SIMD capability does not change the primary classification of the complete multicore system as MIMD.

---

### 6.3 Evidence and Classification Table

| Evidence                                                     | Observation from Task 1 / Task 2                        |
|--------------------------------------------------------------|---------------------------------------------------------|
| Physical core count                                          | 12 physical cores                                       |
| Independent instruction streams observed (Task 2 threads)    | 12 independent worker threads were created and executed |
| SIMD instruction-set flags present (e.g. sse4_2, avx2, neon) | 00                                                      |
| Primary Flynn classification for the machine as a whole      | MIMD — Multiple Instruction, Multiple Data              |

|                                                                                |                                                                              |
|--------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Flynn classification of a single core in isolation (considering its SIMD unit) | SIMD capability, if the corresponding SIMD instruction-set flags are present |
|--------------------------------------------------------------------------------|------------------------------------------------------------------------------|

---

## 6.4 Discussion Questions

### Question 1

**Why is a modern multicore CPU generally classified as MIMD at the machine level, even though each individual core may also execute SIMD instructions internally?**

A modern multicore CPU is generally classified as **MIMD** because it contains multiple processing cores that can execute different instruction streams independently. For example, one core can execute instructions belonging to one thread while another core executes instructions belonging to a different thread. These threads can also operate on different data.

At the same time, an individual core may contain SIMD hardware that allows one instruction to operate on multiple data elements simultaneously.

Therefore:

- **MIMD** describes the behavior of the multicore processor as a whole.
  - **SIMD** describes a type of parallel operation that can occur within an individual core.
- 

### Question 2

**Explain, in your own words, why MISD is rarely seen in general-purpose computing and where it is occasionally used.**

MISD is rarely used in general-purpose computing because most applications benefit more from either executing different tasks independently or applying the same operation to many data elements. Having multiple instruction streams process exactly the same data does not provide a common advantage for most ordinary applications.

MISD can be useful in specialized systems where the same input needs to be processed through different computational methods. It may also be associated with certain fault-tolerant or redundant computing systems, where multiple processing units independently process the same information and their results can be compared.

Therefore, MISD is mainly associated with specialized applications rather than common general-purpose computer systems.

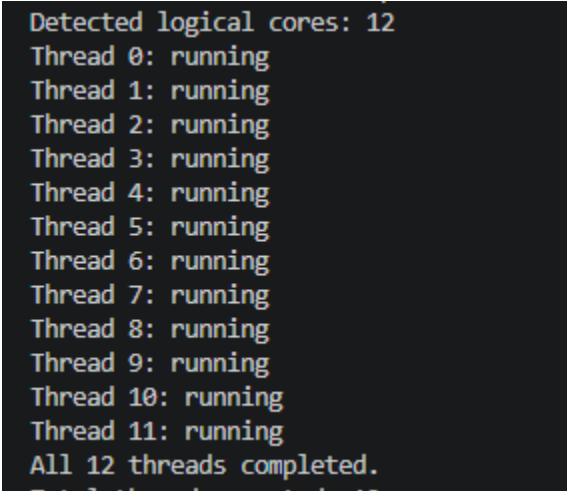
---

### Question 3

**Relate your Task 2 program to the MIMD classification: how does spawning one independent C# thread per core, each capable of executing different code on different data, demonstrate MIMD behaviour in practice?**

The Task 2 program demonstrates MIMD behaviour because it creates multiple independent threads.

The program detected:

A terminal window with a black background and white text. The text shows the detection of 12 logical cores and the subsequent spawning of 12 threads, each labeled 'Thread 0' through 'Thread 11', all in a 'running' state. Finally, a message states 'All 12 threads completed.'

```
Detected logical cores: 12
Thread 0: running
Thread 1: running
Thread 2: running
Thread 3: running
Thread 4: running
Thread 5: running
Thread 6: running
Thread 7: running
Thread 8: running
Thread 9: running
Thread 10: running
Thread 11: running
All 12 threads completed.
```

Each thread represents an independent execution stream. In a real parallel application, these threads could perform different operations or process different portions of a dataset at the same time.

Therefore, the Task 2 experiment provides a practical demonstration of the **Multiple Instruction, Multiple Data (MIMD)** concept.

---

### Question 4

**If the lab machine's cores support AVX/SSE/NEON, give one concrete example of a computation that could exploit that SIMD capability, and briefly explain why.**

One example is **vector addition**. Suppose two arrays contain numerical values:

$A = [1, 2, 3, 4]$

$B = [5, 6, 7, 8]$

The required operation is:

$C = A + B$

$C = [6, 8, 10, 12]$

Without SIMD, the processor may process the elements individually:

$1 + 5$

$2 + 6$

$3 + 7$

$4 + 8$

With SIMD, a suitable vector instruction can operate on several elements simultaneously.

Therefore, multiple additions can be performed as part of one vector operation. This can improve performance for calculations involving large amounts of similar numerical data.

SIMD can be useful in workloads such as:

- Image processing
- Matrix operations
- Audio processing
- Scientific calculations
- Large numerical arrays

### **Lab Task 04: Shared-Memory vs. Distributed-Memory Implications**

The practical implications of shared-memory parallelism on the laboratory machine can be compared with distributed-memory parallelism across a hypothetical cluster consisting of multiple identical machines.

The laboratory machine provides a shared-memory environment in which multiple threads execute within the same operating-system process and share a common virtual address space. In contrast, a distributed-memory cluster consists of multiple independent machines, where each machine has its own processor, operating system, and physical memory.

---

## 7.1 Shared-Memory Model (This Machine)

The laboratory machine used in Tasks 1–3 provides a shared-memory parallel environment. It has **12 physical cores and 12 logical processors**, and the Task 2 program created **12 worker threads**. All threads within the same process can access the same virtual address space and communicate through shared variables.

### Communication Cost

Shared-memory communication is generally very fast because it does not require network communication. However, performance can be affected by cache coherence, memory bandwidth, cache contention, and NUMA effects.

### Synchronization

When multiple threads access shared data, synchronization is required to prevent data races. C# provides mechanisms such as:

- `lock`
- `Monitor`
- `Interlocked`
- `ManualResetEvent`

### Scalability Limit

Shared-memory parallelism is limited by the resources of a single machine, including its cores, memory bandwidth, and cache capacity. In this lab, the Task 2 program used **12 threads** based on the detected **12 logical processors**.

### Fault Model

Threads normally belong to the same process. Therefore, an unhandled process failure can affect all threads running within that process and may terminate the entire application.

---

## 7.2 Distributed-Memory Model (Hypothetical Cluster)

A hypothetical distributed-memory cluster consists of several machines identical to the laboratory machine. Each machine acts as an independent node with its own processor, memory, operating system, and application processes.

### Communication Cost

Different nodes cannot directly access each other's memory. They communicate through a

network using methods such as:

- MPI
- gRPC
- Network sockets
- Message queues

Network communication is generally slower than accessing memory within the same machine, with latency typically ranging from microseconds to milliseconds.

### Synchronization

Distributed systems cannot use a normal shared-memory **lock** between separate machines. Instead, synchronization is performed using:

- Message passing
- Send/receive operations
- Barriers
- Collective communication

### Scalability

Additional machines can be added to increase processing capacity. If one machine has **12 logical processors**, a 4-node cluster could provide approximately:

$$4 \times 12 = 48 \text{ logical processors}$$

This allows the system to scale beyond the resources of one machine.

### Fault Model

Each node operates independently, so failure of one node does not necessarily stop the other nodes. However, the application must detect the failure and handle the work or data belonging to the failed node.

---

## 7.3 Required Steps

### Step 1: Work Division on the Shared-Memory Machine

The Task 2 program detected **12 logical processors** and created **12 worker threads**.

A shared-memory workload could therefore be divided among these threads. For example, if a large dataset contains 1,200,000 records, the workload could theoretically be divided into

approximately 100,000 records per thread:

**$1,200,000 \div 12 = 100,000$  records per thread**

All threads would execute within the same process and could access shared data structures when required.

If multiple threads need to update the same shared variable, synchronization such as **lock** or **Interlocked** would be required to prevent race conditions.

---

### Step 2: Work Division Across N Distributed Nodes

In a hypothetical distributed-memory cluster, the workload would first be divided among the available nodes.

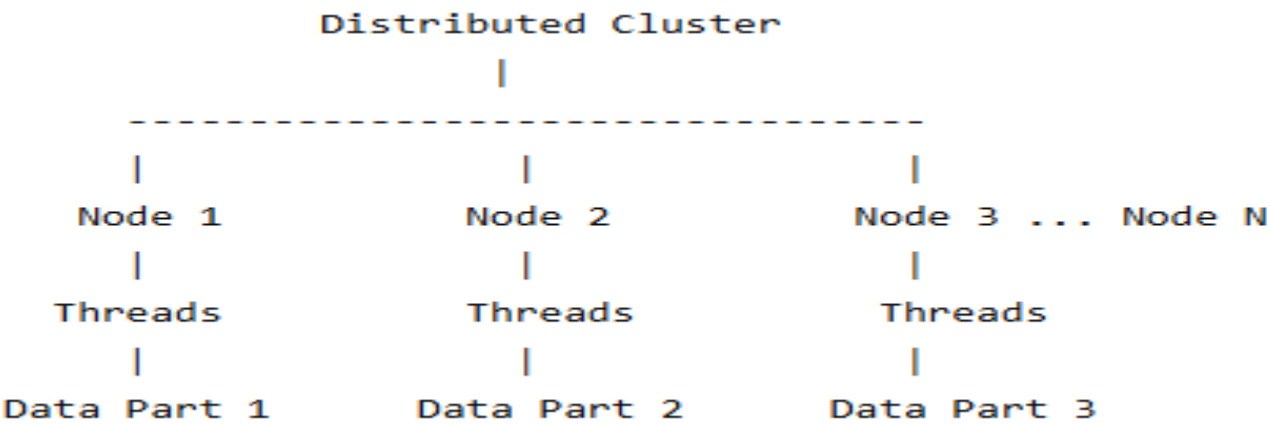
If there are **N identical nodes**, and each node has 12 logical processors, the theoretical number of logical processors would be:

**$N \times 12$**

For example, with 4 nodes:

**$4 \times 12 = 48$  logical processors**

A large dataset could be divided into N major partitions. Each node would process its assigned partition using its local threads



| Aspect                                                              | Shared-Memory (This Machine)                                                                                                 | Distributed-Memory (Hypothetical Cluster)                                                             |
|---------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <b>Address space</b>                                                | Shared virtual address space within the process                                                                              | Each node has its own private memory/address space                                                    |
| <b>Typical communication mechanism</b>                              | Shared variables, memory access, shared data structures                                                                      | Message passing over a network using MPI, gRPC, sockets, or message queues                            |
| <b>Approximate communication latency</b>                            | Typically nanoseconds for memory access, depending on cache/memory level                                                     | Typically microseconds to milliseconds for network communication, depending on network and workload   |
| <b>Synchronization primitives used</b>                              | <code>lock</code> , <code>Monitor</code> , <code>Interlocked</code> , <code>ManualResetEvent</code>                          | Message passing, barriers, collective operations, distributed synchronization                         |
| <b>Maximum practical scale (cores/nodes)</b>                        | Limited by the cores, memory bandwidth, cache, and hardware resources of one machine; this machine has 12 logical processors | Can scale by adding more nodes; for N identical nodes, approximately $N \times 12$ logical processors |
| <b>Effect of a single-unit failure</b>                              | A process failure can affect all threads within that process                                                                 | A node can fail independently, but its work/data must be detected and recovered or reassigned         |
| <b>Programming model / API used in this lab (or MPI equivalent)</b> | C#<br><code>System.Threading.Thread</code> ,<br><code>Thread.Start()</code> ,<br><code>Thread.Join()</code>                  | MPI-style message passing or other distributed communication APIs                                     |

## 7.5 Discussion Questions

### Question 1

If the workload from Task 2 (one unit of work per core) were scaled up to run across a 4-node cluster of identical machines, what new source of overhead would appear

**that was not present when all threads ran on a single machine?**

The major new source of overhead would be **network communication**.

In Task 2, all 12 threads operated on the same machine. Communication between threads could use shared memory, and synchronization could be performed using mechanisms such as **lock** or **Monitor**. With a 4-node cluster, the workload would be divided among four separate machines. If one node needed data or results from another node, that information would have to be transferred through the network.

This introduces additional overhead such as:

- Network latency
- Data transmission time
- Message serialization and deserialization
- Network bandwidth limitations
- Distributed synchronization
- Possible waiting for remote nodes
- Handling communication failures

However, the additional processing resources come with the overhead of communication between nodes.

---

## Question 2

**NUMA effects were recorded in Task 1. Explain why NUMA can be viewed as an intermediate case between pure shared-memory and pure distributed-memory: what does a remote NUMA-node memory access have in common with a network message, and what is fundamentally different about it?**

NUMA, or **Non-Uniform Memory Access**, can be considered an intermediate case because all memory is still part of one shared address space, but accessing some memory regions can take longer than accessing local memory. A thread running on one NUMA node may access memory physically associated with another NUMA node. This remote memory access can have higher latency and may consume additional interconnect bandwidth. This is similar to distributed memory because:

- Access to remote resources has higher latency.
- Communication must travel through an interconnect.
- Data movement can become a performance bottleneck.

However, NUMA is fundamentally different from a distributed-memory cluster because all

NUMA nodes are still part of the **same physical machine** and normally share a single operating-system environment and global virtual address space.

In a distributed-memory cluster:

- Each machine has its own physical memory.
- Each node has its own operating-system environment.
- One node cannot directly dereference another node's memory.
- Data must be explicitly communicated through the network.

Therefore, NUMA provides a shared-memory model with **non-uniform memory access costs**, whereas a distributed cluster provides separate memory spaces requiring explicit communication.

---

### Question 3

**For a computation with very frequent, fine-grained communication between sub-tasks, such as updating a shared value on every iteration of a loop, would you expect this machine's shared-memory model or a distributed-memory cluster to perform better? Justify your answer using the latency figures from your comparison table.**

For a computation involving very frequent and fine-grained communication, the **shared-memory model** is generally more suitable because communication through local memory is significantly lower latency than communication across a network.

The comparison table gives approximate communication scales of:

- **Shared-memory access:** nanoseconds
- **Network communication:** microseconds to milliseconds

Therefore, repeatedly sending small messages between distributed nodes would introduce significant communication overhead. Therefore, for highly communication-intensive and fine-grained workloads, the shared-memory model avoids much of the network communication overhead.

---

### Question 4

**For a computation that can be split into large, mostly-independent chunks of work, such as processing millions of independent records, would a distributed-memory**

**cluster of several such machines be a reasonable way to scale beyond what this single machine can do? Explain why.**

Yes. A distributed-memory cluster can be a suitable approach for workloads that can be divided into large, mostly-independent chunks with little communication between them.

For example, suppose a dataset contains millions of independent records. Each node can process its own partition independently using its local processing cores. If each machine has 12 logical processors, then a 4-node cluster would theoretically provide:

**$4 \times 12 = 48$  logical processors**

The amount of available processing resources can therefore be increased by adding additional machines.

Therefore, workloads such as large-scale data processing, independent record analysis, batch processing, and some scientific computations can be suitable for distributed-memory systems when the work can be divided into sufficiently large independent tasks.

---